

Iniziato venerdì, 13 febbraio 2026, 15:05

Stato Completato

Terminato venerdì, 13 febbraio 2026, 16:07

Tempo impiegato 1 ora 1 min.

Domanda **1**

Completo

Punteggio max.: 4,67

Spiegare perché nelle applicazioni server-side è importante il concetto di "**sessione**" e come esso viene implementato tramite i **cookie**.

quando un utente attraverso client-side genera una richiesta verso server-side, per problemi di stateless di HTTP, nella seconda richiesta server-side dimentica chi era utente , per esempio in un sito di vendita come Adidas.com, quando utente aggiunge un prodotto in cartella e poi naviga sul sito, la pagina si deve ricordare e memorizzare quel prodotto in cartella, questi gestisce da Cookie e session.

quindi nella prima richiesta HTTP da client-side, dopo aver ricevuto la richiesta HTTP da server-side genera un Cookie che sia un blocco di codice e lo aggiunge nella risposta HTTP verso client-side

e in ogni richiesta HTTP da cliente-side, questo blocco codice rimane sempre fin quando viene chiusa la pagina così server-side ricorda sempre (fin quando pagina web aperta) chi è utente

Domanda **2**

Completo

Punteggio max.: 4,67

Si spieghi la differenza concettuale fra la distinzione "client-side/server-side" e la distinzione "front-end/back-end".

front-end: lato front-end detto anche client-side, è la parte visibile che un utente vede, cioè interfaccia(UI) di una pagina web e se pagina sia interattiva complessa può creare i dati verso server-side(back-end) attraverso una richiesta HTTP

backend: lato back-end detto anche server-side, è il core business e accede ai servizi di persistenza dati che può memorizzare, server-side prendere le richieste HTTP da client-side e può elaborare una risposta e mandarla a client-side attraverso response HTTP, oppure se prendere la richiesta HTTP con method (POST) da client-side, può gestire i dati e memorizzare sulla DB (in spring attraverso annotazione @Entity e JPA)

Domanda **3**

Completo

Punteggio max.: 2,00

In un'applicazione Spring che usa la Java Persistence API (JPA), è possibile annotare un metodo con l'annotazione @Transactional. Quale delle seguenti affermazioni sui metodi @Transactional è **vera**?

- ☐ a. L'annotazione @Transactional garantisce che tutte le entità **managed** su cui il metodo opera restino tali (ossia **managed**) per tutta l'esecuzione del metodo, e le modifiche ad esse si riflettano nel DB quando la transazione termina con successo
- ☐ b. L'annotazione @Transactional fa sì che tutte le entità create al suo interno siano transitorie, ossia restino salvate nel DB solo per la durata dell'esecuzione del metodo, per venire poi eliminate quando esso termina
- ☐ c. L'annotazione @Transactional fa sì che tutte le entità salvate nel DB all'interno del metodo vengano salvate con un nuovo **id**, a prescindere che ne avessero o meno uno in precedenza
- ☒ d. L'annotazione @Transactional fa sì che al termine dell'esecuzione del metodo un'eventuale entità restituita continui ad essere **managed** anche dopo che il metodo è terminato.

Domanda **4**

Completo

Punteggio max.: 2,00

In una transazione HTTP, qual è il ruolo del numero di porta all'interno della URL?

- ☐ a. Permettere al client di identificarsi univocamente presso il server
- ☐ b. Permettere a Spring di indirizzare la richiesta HTTP al Controller giusto
- ☒ c. Permettere all'host ricevente di consegnare la richiesta HTTP al servizio corrispondente alla porta specificata
- ☐ d. Permettere al browser di determinare come interpretare e visualizzare la risposta HTTP ricevuta

Domanda **5**

Completo

Punteggio max.: 4,66

Descrivere le fasi che compongono un ciclo del browser, a partire da quando esso ottiene una pagina HTML in risposta a una transazione HTTP precedentemente iniziata

- il browser riceve la pagina HTML attraverso URL
- il browser viene in attesa per ricevere le risorse di visualizzazione (cioè la view) per Renderizza la pagina, sia risorse CSS che un image.
- quando arrivano le risorse, il browser inizia a renderizza la pagina
- e poi vengono gli scrip e lo esegue e se la pagina sia interattiva:
 - in pagina interattiva utente può generare gli eventi esogeni sullo Script
- si genera un evento attraverso gli scrip lo gestisce
- se a causa di evento, cambia lo style della pagina, allora il browser prende nuove risorse e inizia a re-renderizza la pagina

Domanda **6**

Completo

Punteggio max.: 2,00

Le applicazioni web server-side realizzate con Spring Web sono basate sul pattern Model-View-Controller. Per ciascuno dei seguenti elementi costitutivi di un'applicazione Spring Web stabilisci se fa parte del Model, della View, del Controller o di nessuno dei tre.

body della HTTP Request	Nessuno dei tre
body della HTTP Response	View
@Entity	Model
@Service	Model
@RestController	Controller

Domanda **7**

Completo

Punteggio max.: 2,00

Si consideri il seguente codice Javascript (le righe sono numerate per comodità):

```
1.    const gatto = {  
2.        nome: "Annibale",  
3.        colore: "Grigio"  
4.    };  
5.    gatto.anni = 5;
```

L'istruzione alla riga 5 modifica l'oggetto *gatto*, che era dichiarato usando la parola chiave **const**. È corretta o dà luogo ad un errore? Per quale ragione?

- ☐ a. Dà errore perché la dichiarazione *const* ammette che si modifichi il valore una proprietà dell'oggetto da essa riferito, ma non che si aggiungano nuove proprietà o se ne rimuovano di esistenti.
- ☐ b. È corretta perché l'istruzione alla riga 5 aggiunge una nuova proprietà all'oggetto (la proprietà *anni*); se modificasse una proprietà esistente invece darebbe errore.
- ☒ c. È corretta perché *gatto* contiene un riferimento all'oggetto, che non viene modificato dall'istruzione alla riga 5. Se l'istruzione assegnasse a *gatto* un nuovo oggetto, darebbe errore.
- ☐ d. Dà errore perché la dichiarazione *const* implica che il valore assegnato a *gatto* sia immutabile e dunque *gatto* non possa mai comparire sulla sinistra di un assegnamento.

Domanda **8**

Completo

Punteggio max.: 2,00

Cos'è un **request handler** in Spring?

- ☐ a. Un'annotazione che va premessa a tutte le classi che fungono da controller, ossia sono atte a ricevere richieste HTTP
- ☐ b. Un oggetto che rappresenta una richiesta HTTP per un endpoint corrispondente a una route parametrica
- ☐ c. Una classe i cui metodi permettono di ricevere richieste HTTP per tutti gli endpoint con un determinato prefisso
- ☒ d. Un metodo responsabile di ricevere richieste HTTP per un determinato endpoint (o sottoinsieme di endpoint)

Domanda **9**

Completo

Punteggio max.: 2,00

Un componente React è definito come una funzione, detta **funzione di rendering**.
Cosa deve restituire tale funzione?

- ☐ a. una stringa contenente codice HTML puro
- ☒ b. Un'espressione JSX o TSX
- ☐ c. Un oggetto JSON
- ☐ d. Una funzione di callback

Domanda **10**

Completo

Punteggio max.: 2,00

Parliamo di dichiarazioni di **tipo** in Typescript. Dobbiamo definire una variabile **x** atta a contenere un **array di esattamente tre elementi**, dei quali il primo deve essere un **numero**, il secondo può essere un **numero o una stringa**, e il terzo può essere soltanto "verificato" oppure

- il primo deve essere un numero
- il secondo può essere un numero oppure una stringa
- il terzo può avere solo uno fra i due valori "verificato" oppure "in corso di verifica"

Si indichi come deve essere dichiarata x per ottenere quanto desiderato.

- ☐ a. `let x: (number, number | string, "verificato" | "in corso di verifica")[3]`
- ☐ b. `let x: [number | string][3];`
- ☒ c. `let x: [number, number | string, "verificato" | "in corso di verifica"];`
-
- ☐ d. `let x: (number | string)[]`
- ☐ e. `let x: [number, number | string, string];`
- ☐ f. `let x: [number, number, string, "verificato", "in corso di verifica"];`

Domanda **11**

Completo

Punteggio max.: 2,00

Quale delle seguenti affermazioni sugli **oggetti in Javascript** è vera?

- ☒ a. Un oggetto può perdere alcune delle proprietà che aveva al momento della creazione
- ☐ b. Per qualunque oggetto *ogg*, vale che il suo prototipo diretto *ogg.__proto__* coincide con *Object.prototype*
- ☐ c. Di default, le proprietà di un oggetto sono private e non possono essere viste al di fuori dei metodi dell'oggetto
- ☐ d. Un oggetto eredita dal prototipo i metodi, ma non le proprietà

Domanda **12**

Completo

Punteggio max.: 2,00

Un componente React è caratterizzato, fra le altre cose, dal suo «**stato**». Quale di queste affermazioni sullo **stato** di un componente React è **vera**?

- ☒ a. Lo **stato** di un componente è costituito da una o più proprietà gestite da React, ad ogni esecuzione della funzione di rendering, viene restituito al componente lo stato corrente e una funzione da invocare per modificarlo
- ☐ b. Lo **stato** del componente non può essere definito nella funzione di rendering, ma può solo essere definito all'interno di funzioni particolari chiamati "effect", che vengono eseguite in modo asincrono rispetto al rendering.
- ☐ c. Lo **stato** di un componente può essere modificato solo ed esclusivamente in conseguenza di eventi UI.
- ☐ d. La modifica allo **stato** del componente provoca un re-rendering solo se una o più delle variabili di stato vengono interpolate nell'espressione JSX/TSX che definisce la view del componente.